B2B Travel Booking & Revenue Analytics

PHASE 2 - STAGING & CLEAN DATA LAYERS

Complete Beginner-Friendly Documentation

What to do | How to do it | Why to do it | Concepts | SQL | Validation | Analyst thinking

Core Phase 2 idea Raw preserves the source. Staging prepares and standardizes it. Clean contains trusted, analysis-ready records.

Project stack: PostgreSQL / pgAdmin | SQL | Python | Power BI | Excel

Learning mode: beginner-friendly, step-by-step

1. Phase 2 Overview

Phase 2 converts the profiled raw data into controlled staging and clean layers. The goal is not simply to “delete bad rows.” The goal is to preserve the original source, standardize data types and categories, document exceptions, remove only records that are unsafe for core KPIs, and create tables at reliable business grains.

What Phase 2 answers

- How do we convert text dates and numeric fields safely?
- How do we standardize country names, statuses, currencies, categories and flags?
- How do we remove duplicate booking copies without touching the raw layer?
- Which data-quality problems should exclude a booking from KPI analysis?
- Which missing values should be flagged rather than deleted?
- How do we build fact and dimension tables with controlled grains?
- How do we prevent one-to-many payment joins from double counting revenue?
- How do we validate the clean layer before moving to FX normalization?

Check	Verified result	Meaning
Raw booking rows	120,175	Original source rows, including duplicates and quality issues
Unique booking IDs after deduplication	120,000	One row candidate per booking
Serious invalid unique bookings	123	Rooms/nights/GBV/date-sequence issues
Final clean fact_bookings	119,877	Unique bookings safe for the main KPI layer
Booking DQ exception table	123	Serious invalid records preserved for traceability
Missing supplier IDs	240	Kept and flagged because booking can still support many analyses
Verified missing agent references	0	Earlier conflicting result was reconciled
Clean cancellation rows	13,972	13,987 raw cancellations minus 15 true orphans

Important Phase 2 monetary values are still in transaction currencies. Do not sum company-wide GBV or revenue across currencies yet. FX normalization belongs to Phase 3.

2. The Layered Architecture

Pipeline RAW -> STAGING -> CLEAN -> ANALYTICS

Layer	Purpose	Analyst rule
RAW	Original imported data	Preserve source exactly; useful for audit and reprocessing
STAGING	Typed, standardized, flagged data	Prepare data without pretending every row is trustworthy
CLEAN	Trusted facts, dimensions and exception tables	Create stable inputs for KPIs, analytics and BI
ANALYTICS	Aggregated marts and business metrics	Built later after FX normalization and controlled joins

Why raw data stays unchanged

If you overwrite the raw table, you lose the ability to prove what arrived from the source. Keeping raw data unchanged gives you traceability, makes debugging easier, and lets you rebuild staging and clean tables when business rules change.

3. Lessons 1-3: Build and Validate the Staging Foundation

Lesson 1: Create the staging and clean schemas

WHAT TO DO	WHY	HOW
Create two new PostgreSQL schemas: staging and clean.	Separate preparation logic from trusted analytical data and keep raw source tables untouched.	Use CREATE SCHEMA IF NOT EXISTS so rerunning the setup is safe.

```
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS clean;
```

Concept: schema A schema is like a folder inside a PostgreSQL database. It organizes tables by purpose.

Lesson 2: Create staging.bookings_typed

WHAT TO DO	WHY	HOW
Copy raw bookings into staging while adding real DATE and NUMERIC helper columns.	Raw CSV imports often store dates and numbers as text. Analysis is safer after explicit type conversion.	Use CASE, regex pattern checks, ::date, TO_DATE() and ::numeric. Keep the original columns for traceability.

```
CREATE TABLE staging.bookings_typed AS
SELECT
    b.*,
    CASE
        WHEN booking_date ~ '^\d{4}-\d{2}-\d{2}$' THEN booking_date::date
        WHEN booking_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(booking_date,'DD/MM/YYYY')
        ELSE NULL
    END AS booking_dt,
    CASE
        WHEN travel_date ~ '^\d{4}-\d{2}-\d{2}$' THEN travel_date::date
        WHEN travel_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(travel_date,'DD/MM/YYYY')
        ELSE NULL
    END AS travel_dt,
    CASE WHEN TRIM(rooms) ~ '^-?\d+(\.\d+)?$' THEN rooms::numeric END AS rooms_num,
    CASE WHEN TRIM(nights) ~ '^-?\d+(\.\d+)?$' THEN nights::numeric END AS nights_num,
    CASE WHEN TRIM(gross_booking_value) ~ '^-?\d+(\.\d+)?$' THEN gross_booking_value::numeric END AS gbv_num,
    CASE WHEN TRIM(supplier_cost) ~ '^-?\d+(\.\d+)?$' THEN supplier_cost::numeric END AS supplier_cost_num,
    CASE WHEN TRIM(discount_amount) ~ '^-?\d+(\.\d+)?$' THEN discount_amount::numeric END AS discount_num
FROM raw.bookings AS b;
```

The regex check protects the cast. If a raw value were “N/A” or “two”, the CASE can return NULL instead of crashing the whole statement.

Check	Verified result	Meaning
staging.bookings_typed row count	120,175	Same as raw because staging has not removed records yet

Lesson 3: Validate the typed columns

WHAT TO DO	WHY	HOW
Count NULLs created in the new typed columns.	A table can be created successfully even if conversions silently failed. Validation is what makes a transformation trustworthy.	Use COUNT(*) FILTER (WHERE ... IS NULL) for each typed helper column.

```
SELECT
    COUNT(*) FILTER (WHERE booking_dt IS NULL) AS null_booking_dt,
    COUNT(*) FILTER (WHERE travel_dt IS NULL) AS null_travel_dt,
    COUNT(*) FILTER (WHERE rooms_num IS NULL) AS null_rooms_num,
    COUNT(*) FILTER (WHERE nights_num IS NULL) AS null_nights_num,
    COUNT(*) FILTER (WHERE gbv_num IS NULL) AS null_gbv_num,
    COUNT(*) FILTER (WHERE supplier_cost_num IS NULL) AS null_supplier_cost_num,
    COUNT(*) FILTER (WHERE discount_num IS NULL) AS null_discount_num
FROM staging.bookings_typed;
```

Check	Verified result	Meaning
booking_dt conversion failures	0	All booking dates converted
travel_dt conversion failures	0	All travel dates converted
rooms_num conversion failures	0	All rooms values converted
nights_num conversion failures	0	All nights values converted
gbv_num conversion failures	0	All GBV values converted
supplier_cost_num failures	0	All supplier costs converted
discount_num failures	0	All discounts converted

Analyst habit Transform -> Validate -> Trust. Never skip the validation step.

4. Lessons 4-5: Standardize Categorical Text

Lesson 4: Standardize travel-agent country names

WHAT TO DO	WHY	HOW
Create standardized country values while preserving the original country field.	SQL treats India, INDIA and india as different text values. That would split reports into false categories.	Use TRIM(), UPPER(), INITCAP() and CASE for known aliases or spelling corrections.

```
CREATE TABLE staging.travel_agents_std AS
SELECT
    a.*,
    CASE
        WHEN UPPER(TRIM(country)) IN ('UAE','U.A.E','U.A.E.','UNITED ARAB EMIRATES')
            THEN 'United Arab Emirates'
        WHEN UPPER(TRIM(country)) = 'SIGNAPORE' THEN 'Singapore'
        ELSE INITCAP(TRIM(country))
    END AS country_std
FROM raw.travel_agents AS a;
```

TRIM removes spaces. UPPER gives a reliable comparison form. CASE applies known business mappings. INITCAP produces readable final labels.

Lesson 5: Standardize booking status, payment status and currency

WHAT TO DO	WHY	HOW
Create standardized status and currency columns.	Category inconsistencies create duplicate labels in SQL and Power BI. Currency codes should use a consistent uppercase convention.	Use INITCAP(TRIM(...)) for status labels and UPPER(TRIM(...)) for currency codes.

```
CREATE TABLE staging.bookings_std AS
SELECT
    b.*,
    INITCAP(TRIM(booking_status)) AS booking_status_std,
    INITCAP(TRIM(payment_status)) AS payment_status_std,
    UPPER(TRIM(currency)) AS currency_std
FROM staging.bookings_typed AS b;
```

Check	Verified result	Meaning
booking_status_std	Cancelled, Completed, Confirmed	Three consistent booking lifecycle categories
payment_status_std in bookings	Paid, Refunded	Booking-level payment labels standardized
currency_std	15 three-letter codes	AED, AUD, EGP, EUR, GBP, IDR, INR, JPY, MYR, QAR, SAR, SGD, THB, TRY, USD

Currency caution Standardizing a code does not make currencies comparable. USD + INR is still meaningless until Phase 3 FX conversion.

5. Lessons 6-8: Deduplicate the Booking Grain

Lesson 6: Understand ROW_NUMBER() for duplicate ranking

WHAT TO DO	WHY	HOW
Assign a row number within each booking_id group.	You need a repeatable way to identify the first copy and extra copies of a booking.	Use ROW_NUMBER() OVER (PARTITION BY booking_id ORDER BY ...).

```
SELECT
    booking_id,
    ROW_NUMBER() OVER (
        PARTITION BY booking_id
        ORDER BY booking_id
    ) AS duplicate_rank
FROM staging.bookings_std
WHERE booking_id = 'B0000944';
```

Check	Verified result	Meaning
B0000944 first copy	duplicate_rank = 1	Keep candidate
B0000944 second copy	duplicate_rank = 2	Extra duplicate copy

Concept: PARTITION BY It restarts the row numbering for each booking_id. A unique booking gets rank 1; a duplicated booking gets ranks 1, 2, ...

Lesson 7: Rank duplicates for the entire table

```
CREATE TABLE staging.bookings_ranked AS
SELECT
    b.*,
    ROW_NUMBER() OVER (
        PARTITION BY booking_id
        ORDER BY booking_id
    ) AS duplicate_rank
FROM staging.bookings_std AS b;
```

Check	Verified result	Meaning
Total ranked rows	120,175	Ranking does not remove rows
Rows where duplicate_rank > 1	175	Extra duplicate copies

Lesson 8: Create a deduplicated booking table

WHAT TO DO	WHY	HOW
Keep duplicate_rank = 1.	The analytical booking grain must be one row per booking. Duplicate copies inflate counts and money metrics.	Create a clean intermediate table without touching raw data.

```
CREATE TABLE clean.bookings_deduped AS
SELECT *
FROM staging.bookings_ranked
WHERE duplicate_rank = 1;
```

Check	Verified result	Meaning
Total rows	120,000	One row per surviving booking
Unique booking IDs	120,000	Booking grain is controlled
Duplicate rows	0	No duplicated booking IDs remain

6. Lessons 9-13: Data-Quality Flags and Exceptions

Lesson 9: Create a simple invalid-rooms flag

WHAT TO DO	WHY	HOW
Mark invalid room quantities with 1 and valid rows with 0.	Flags explain why a row is problematic without immediately deleting it.	Use CASE WHEN rooms_num <= 0 THEN 1 ELSE 0 END.

```
CASE
    WHEN rooms_num <= 0 THEN 1
    ELSE 0
END AS invalid_rooms
```

Check	Verified result	Meaning
Invalid room bookings after deduplication	20	Same count remained after duplicate removal

Lesson 10: Create multiple serious business-rule flags

```
SELECT
    COUNT(*) FILTER (WHERE rooms_num <= 0) AS invalid_rooms,
    COUNT(*) FILTER (WHERE nights_num <= 0) AS invalid_nights,
    COUNT(*) FILTER (WHERE gbv_num <= 0) AS invalid_gbv,
    COUNT(*) FILTER (WHERE travel_dt < booking_dt) AS invalid_date_sequence
FROM clean.bookings_deduped;
```

Check	Verified result	Meaning
Invalid rooms	20	rooms <= 0
Invalid nights	20	nights <= 0
Invalid GBV	58	gross booking value <= 0
Invalid date sequence	25	travel date before booking date

Lesson 11: Count unique bookings with any serious issue

WHAT TO DO	WHY	HOW
Count bookings affected by at least one serious rule.	Adding issue counts can overstate affected records when one row has multiple problems.	Use OR between conditions so each booking row is counted once.

```
SELECT COUNT(*) AS bookings_with_any_serious_issue
FROM clean.bookings_deduped
WHERE rooms_num <= 0
   OR nights_num <= 0
   OR gbv_num <= 0
   OR travel_dt < booking_dt;
```

Check	Verified result	Meaning
Bookings with any serious issue	123	In this dataset, the four serious issue groups did not overlap after deduplication

Lesson 12: Create the data-quality exceptions table

WHAT TO DO	WHY	HOW

Store serious invalid bookings separately with issue flags.	Excluded rows must remain explainable and auditable. “Bad” data should not simply disappear.	Create clean.booking_data_quality_exceptions from the deduplicated booking set.

```
CREATE TABLE clean.booking_data_quality_exceptions AS
SELECT
    booking_id, booking_dt, travel_dt, rooms_num, nights_num, gbv_num,
    CASE WHEN rooms_num <= 0 THEN 1 ELSE 0 END AS invalid_rooms,
    CASE WHEN nights_num <= 0 THEN 1 ELSE 0 END AS invalid_nights,
    CASE WHEN gbv_num <= 0 THEN 1 ELSE 0 END AS invalid_gbv,
    CASE WHEN travel_dt < booking_dt THEN 1 ELSE 0 END AS invalid_date_sequence
FROM clean.bookings_deduped
WHERE rooms_num <= 0
   OR nights_num <= 0
   OR gbv_num <= 0
   OR travel_dt < booking_dt;
```

Check	Verified result	Meaning
Exception rows	123	Serious invalid bookings preserved for investigation

Lesson 13: Handle missing suppliers and reconcile agent references

WHAT TO DO	WHY	HOW
Differentiate a structurally invalid booking from an incomplete dimension reference.	A missing supplier does not make the entire booking unusable. It mainly limits supplier-level analysis.	Flag missing suppliers; validate nonblank foreign keys with LEFT JOIN; reconcile conflicting counts before moving on.

```
SELECT
    COUNT(*) FILTER (
        WHERE b.supplier_id IS NULL OR TRIM(b.supplier_id) = ''
    ) AS missing_supplier_bookings,
    COUNT(*) FILTER (
        WHERE a.agent_id IS NULL
    ) AS missing_agent_reference_bookings
FROM clean.bookings_deduped AS b
LEFT JOIN staging.travel_agents_std AS a
    ON b.agent_id = a.agent_id;
```

Check	Verified result	Meaning
Missing supplier bookings	240	Keep and flag; exclude only from supplier-specific analysis when needed
Verified missing agent references	0	Earlier Phase 1 value of 1,000 was rechecked and corrected after reconciliation

Reconciliation lesson When yesterday’s number and today’s number disagree, stop and investigate. Do not choose the number you prefer.

7. Lesson 14: Build clean.fact_bookings

WHAT TO DO	WHY	HOW
Create the main analysis-ready booking fact table.	Core KPIs need one valid row per booking with trusted dates, quantities and GBV.	Keep deduplicated rows that pass serious business rules. Preserve missing supplier IDs as a flag.

```
CREATE TABLE clean.fact_bookings AS
SELECT
    booking_id,
    booking_dt,
    travel_dt,
    agent_id,
    hotel_id,
    supplier_id,
    destination_id,
    rooms_num AS rooms,
    nights_num AS nights,
    booking_status_std AS booking_status,
    payment_status_std AS payment_status,
    currency_std AS currency,
    gbv_num AS gross_booking_value,
    supplier_cost_num AS supplier_cost,
    discount_num AS discount_amount,
    travel_dt - booking_dt AS lead_days,
    gbv_num - supplier_cost_num - discount_num AS net_revenue,
    ROUND(((gbv_num - supplier_cost_num - discount_num)
        / NULLIF(gbv_num,0)) * 100, 2) AS gross_margin_pct,
    CASE WHEN booking_status_std = 'Cancelled' THEN 1 ELSE 0 END AS is_cancelled,
    CASE WHEN supplier_id IS NULL OR TRIM(supplier_id) = '' THEN 1 ELSE 0 END AS is_supplier_missing
FROM clean.bookings_deduped
WHERE rooms_num > 0
  AND nights_num > 0
  AND gbv_num > 0
  AND travel_dt >= booking_dt;
```

Calculated fields

Field	Formula / rule	Why useful
lead_days	travel_dt - booking_dt	How far in advance the booking was made
net_revenue	GBV - supplier cost - discount	Commercial contribution before later reporting adjustments
gross_margin_pct	net_revenue / GBV x 100	Booking-level margin percentage
is_cancelled	1 for Cancelled, else 0	Easy cancellation-rate calculations
is_supplier_missing	1 when supplier is blank/NULL	Preserves booking while warning supplier analysis

NULLIF() NULLIF(gbv_num, 0) protects division from a zero denominator. Defensive SQL is useful even when the WHERE clause already requires GBV > 0.

Check	Verified result	Meaning
Final fact rows	119,877	120,000 unique bookings - 123 serious invalid bookings
Unique booking IDs	119,877	Exactly one row per valid booking
Duplicate rows	0	Fact grain is controlled
Invalid rooms/nights/GBV/date rows	0 / 0 / 0 / 0	Serious rules are clean in the final fact

8. Lesson 15: Build Clean Dimension Tables

A fact table tells you what happened. Dimension tables describe who, what and where. Before creating a dimension, always confirm that its key is unique so a join cannot multiply fact rows.

Grain rule fact_bookings: one row per booking. dim_agents: one row per agent. dim_hotels: one row per hotel. dim_suppliers: one row per supplier. dim_destinations: one row per destination.

Lesson 15A-15B: Agent dimension

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT agent_id) AS unique_agent_ids,
    COUNT(*) - COUNT(DISTINCT agent_id) AS duplicate_agent_ids
FROM staging.travel_agents_std;
```

Check	Verified result	Meaning
Agent rows	8,000	Source dimension size
Unique agent IDs	8,000	Correct one-row-per-agent grain
Duplicate agent IDs	0	Safe for joining

```
CREATE TABLE clean.dim_agents AS
SELECT
    agent_id,
    agent_name,
    agent_type,
    country_std AS country
FROM staging.travel_agents_std;
```

Lesson 15C-15G: Hotel dimension

Hotel profiling checks key uniqueness, hotel category labels, active flags and missing star ratings. Missing ratings are preserved as NULL rather than invented.

Check	Verified result	Meaning
Hotel rows	3,000	Source dimension size
Unique hotel IDs	3,000	Correct grain
Duplicate hotel IDs	0	Safe for joining
Missing star ratings	35	Preserved as NULL
Active flag	Y 2,884 / N 116	Standardized uppercase flag

```
CREATE TABLE staging.hotels_std AS
SELECT
    h.*,
    CASE
        WHEN UPPER(TRIM(hotel_category)) = 'UPWCALE' THEN 'Upscale'
        WHEN UPPER(TRIM(hotel_category)) = 'BUGET' THEN 'Budget'
        ELSE INITCAP(TRIM(hotel_category))
    END AS hotel_category_std,
    UPPER(TRIM(active_flag)) AS active_flag_std,
    CASE
        WHEN star_rating IS NULL OR TRIM(star_rating) = '' THEN NULL
        WHEN TRIM(star_rating) ~ '^\d+(\.\d+)?$' THEN star_rating::numeric
        ELSE NULL
    END AS star_rating_num
FROM raw.hotels AS h;
```

Validation note The source package itself uses clean labels Upscale, Midscale, Luxury and Budget. During the interactive lesson, some category spellings were typed manually and appeared inconsistent. Final portfolio SQL should validate the stored values directly before applying typo-specific CASE rules.

```
CREATE TABLE clean.dim_hotels AS
SELECT
    hotel_id, hotel_name, destination_id,
    star_rating_num AS star_rating,
    hotel_category_std AS hotel_category,
    chain_name,
    active_flag_std AS active_flag
FROM staging.hotels_std;
```

Lesson 15H-15L: Supplier dimension

Check	Verified result	Meaning
Supplier rows	160	Source dimension size
Unique supplier IDs	160	Correct grain
Duplicate supplier IDs	0	Safe for joining
Supplier type distribution	72 + 51 + 37 = 160	All supplier rows accounted for

```
CREATE TABLE staging.suppliers_std AS
SELECT
    s.*,
    INITCAP(TRIM(supplier_country)) AS supplier_country_std,
    CASE
        WHEN UPPER(TRIM(supplier_type)) = 'AGGRETOR' THEN 'Aggregator'
        ELSE INITCAP(TRIM(supplier_type))
    END AS supplier_type_std,
    INITCAP(TRIM(contract_type)) AS contract_type_std,
    CASE WHEN TRIM(commission_rate) ~ '^\d+(\.\d+)?$'
         THEN commission_rate::numeric END AS commission_rate_num
FROM raw.suppliers AS s;
```

Validation note The packaged source uses Aggregator, Hotel Direct and DMC. Use profiling results from the database as the source of truth, not manually retyped labels.

```
CREATE TABLE clean.dim_suppliers AS
SELECT
    supplier_id,
    supplier_name,
    supplier_country_std AS supplier_country,
    supplier_type_std AS supplier_type,
    contract_type_std AS contract_type,
    commission_rate_num AS commission_rate
FROM staging.suppliers_std;
```

Lesson 15M-15P: Destination dimension

Check	Verified result	Meaning
Destination rows	280	Source dimension size
Unique destination IDs	280	Correct grain
Duplicate destination IDs	0	Safe for joining

```
CREATE TABLE staging.destinations_std AS
SELECT
    d.*,
    INITCAP(TRIM(city)) AS city_std,
    INITCAP(TRIM(country)) AS country_std,
    INITCAP(TRIM(region)) AS region_std,
    INITCAP(TRIM(destination_type)) AS destination_type_std
FROM raw.destinations AS d;

CREATE TABLE clean.dim_destinations AS
SELECT
    destination_id,
    city_std AS city,
    country_std AS country,
    region_std AS region,
    destination_type_std AS destination_type
FROM staging.destinations_std;
```

9. Lesson 16: Clean the Cancellations Table

Lesson 16A-16B: Validate grain and convert types

WHAT TO DO	WHY	HOW
Confirm cancellation_id uniqueness, then convert cancellation date, refund amount and fee.	A cancellation is a business event and its monetary/date fields must be typed before analysis.	Use the same safe date and numeric conversion patterns used for bookings.

Check	Verified result	Meaning
Raw cancellation rows	13,987	One row per cancellation event
Unique cancellation IDs	13,987	No duplicate cancellation IDs
Duplicate cancellation IDs	0	Correct event grain
Null typed cancellation dates	0	All dates converted
Null refund amounts after typing	0	All refund amounts converted
Null cancellation fees after typing	0	All fees converted

```
CREATE TABLE staging.cancellations_typed AS
SELECT
    c.*,
    CASE
        WHEN cancellation_date ~ '^\d{4}-\d{2}-\d{2}$' THEN cancellation_date::date
        WHEN cancellation_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(cancellation_date,'DD/MM/YYYY')
        ELSE NULL
    END AS cancellation_dt,
    CASE WHEN TRIM(refund_amount) ~ '^-?\d+(\.\d+)?$' THEN refund_amount::numeric END AS refund_amount_num,
    CASE WHEN TRIM(cancellation_fee) ~ '^-?\d+(\.\d+)?$' THEN cancellation_fee::numeric END AS cancellation_fee_num
FROM raw.cancellations AS c;
```

Lesson 16C-16D: Standardize cancellation categories

```
CREATE TABLE staging.cancellations_std AS
SELECT
    c.*,
    INITCAP(TRIM(cancellation_reason)) AS cancellation_reason_std,
    INITCAP(TRIM(cancelled_by)) AS cancelled_by_std
FROM staging.cancellations_typed AS c;
```

The same principle applies: normalize capitalization and spaces first, then add CASE rules only for genuine aliases or spelling problems found during profiling.

Lesson 16E-16F: Remove true orphan cancellations from the clean fact

WHAT TO DO	WHY	HOW
Identify cancellation booking IDs that have no real booking, then exclude those orphan events from the clean cancellation fact.	A cancellation without a parent booking cannot be safely related to booking metrics.	LEFT JOIN to find orphans; INNER JOIN to keep only known bookings in the clean fact.

Check	Verified result	Meaning
Orphan cancellations	15	Cancellation record exists but booking does not
Clean cancellation rows	13,972	13,987 - 15 orphans
Unique clean cancellation IDs	13,972	One row per clean cancellation
Orphans after cleaning	0	Clean fact has valid parent bookings

```
CREATE TABLE clean.fact_cancellations AS
SELECT
    c.cancellation_id,
    c.booking_id,
    c.cancellation_dt AS cancellation_date,
    c.cancellation_reason_std AS cancellation_reason,
    c.refund_amount_num AS refund_amount,
    c.cancellation_fee_num AS cancellation_fee,
    c.cancelled_by_std AS cancelled_by
FROM staging.cancellations_std AS c
INNER JOIN clean.bookings_deduped AS b
    ON c.booking_id = b.booking_id;
```

10. Lesson 17: Prepare and Aggregate Payments Safely

Payments are the most important grain lesson in Phase 2. The payment table is transaction-level, while the booking fact is booking-level. Joining them directly can repeat the booking GBV and inflate totals.

Critical rule Before joining a one-to-many transaction table to a one-row-per-booking fact, aggregate the many-side to booking grain when your KPI needs one row per booking.

Lesson 17A-17B: Validate payment IDs and type payment fields

Check	Verified result	Meaning
Payment rows in source package	127,189	Transaction-level records
Unique payment IDs	127,189	payment_id is unique
Duplicate payment IDs	0	Correct payment transaction grain
Payment statuses in source	Successful / Refunded / Failed	Transaction-level status values
Payment methods in source	Card / Credit / Wallet	Payment-method categories

```
CREATE TABLE staging.payments_typed AS
SELECT
    p.*,
    CASE
        WHEN payment_date ~ '^\d{4}-\d{2}-\d{2}$' THEN payment_date::date
        WHEN payment_date ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(payment_date,'DD/MM/YYYY')
        ELSE NULL
    END AS payment_dt,
    CASE WHEN TRIM(amount) ~ '^-?\d+(\.\d+)?$' THEN amount::numeric END AS amount_num,
    INITCAP(TRIM(payment_method)) AS payment_method_std,
    INITCAP(TRIM(payment_status)) AS payment_status_std
FROM raw.payments AS p;
```

Lesson 17C: Aggregate payment transactions to booking grain

```
CREATE TABLE clean.payment_booking_summary AS
SELECT
    booking_id,
    COUNT(*) AS payment_rows,
    SUM(amount_num) AS total_payment_amount,
    SUM(CASE WHEN payment_status_std = 'Successful' THEN amount_num ELSE 0 END) AS successful_payment_amount,
    SUM(CASE WHEN payment_status_std = 'Refunded' THEN amount_num ELSE 0 END) AS refunded_amount,
    SUM(CASE WHEN payment_status_std = 'Failed' THEN amount_num ELSE 0 END) AS failed_amount,
    MIN(payment_dt) AS first_payment_date,
    MAX(payment_dt) AS last_payment_date
FROM staging.payments_typed
GROUP BY booking_id;
```

After GROUP BY booking_id, each booking appears once in the summary. This protects booking-level GBV, revenue and count metrics from multiplication when payments are joined later.

Reconciliation note During the earlier interactive lesson, 245 booking IDs were reported as having multiple payment rows. The packaged source contains 127,189 payment rows across 120,000 booking IDs, which implies 7,189 booking IDs have a second payment row. Re-run the GROUP BY/HAVING query in your database before using this count in a portfolio claim.

Lesson 17D: Check orphan payment records

```
SELECT COUNT(*) AS orphan_payment_rows
FROM staging.payments_typed AS p
LEFT JOIN clean.bookings_deduped AS b
    ON p.booking_id = b.booking_id
WHERE b.booking_id IS NULL;
```

Also check the aggregated payment summary against the booking table. Transaction-level orphan rows and orphan booking IDs are different counts and should not be confused.

11. Lesson 18: Final Phase 2 Validation

WHAT TO DO	WHY	HOW
Reconcile row counts, uniqueness, logical business rules and foreign-key relationships.	A clean layer is not trustworthy until you prove the transformations and joins behaved as intended.	Run final count, distinct-key, rule and relationship checks across all clean tables.

```
SELECT
    (SELECT COUNT(*) FROM clean.fact_bookings) AS fact_bookings_rows,
    (SELECT COUNT(*) FROM clean.fact_cancellations) AS fact_cancellations_rows,
    (SELECT COUNT(*) FROM clean.payment_booking_summary) AS payment_summary_rows,
    (SELECT COUNT(*) FROM clean.dim_agents) AS dim_agents_rows,
    (SELECT COUNT(*) FROM clean.dim_hotels) AS dim_hotels_rows,
    (SELECT COUNT(*) FROM clean.dim_suppliers) AS dim_suppliers_rows,
    (SELECT COUNT(*) FROM clean.dim_destinations) AS dim_destinations_rows;
```

```
SELECT
    COUNT(*) FILTER (WHERE rooms <= 0) AS invalid_rooms,
    COUNT(*) FILTER (WHERE nights <= 0) AS invalid_nights,
    COUNT(*) FILTER (WHERE gross_booking_value <= 0) AS invalid_gbv,
    COUNT(*) FILTER (WHERE travel_dt < booking_dt) AS invalid_dates
FROM clean.fact_bookings;
```

```
SELECT
    COUNT(*) FILTER (WHERE a.agent_id IS NULL) AS missing_agents,
    COUNT(*) FILTER (WHERE h.hotel_id IS NULL) AS missing_hotels,
    COUNT(*) FILTER (WHERE d.destination_id IS NULL) AS missing_destinations,
    COUNT(*) FILTER (
        WHERE b.supplier_id IS NOT NULL
          AND TRIM(b.supplier_id) <> ''
          AND s.supplier_id IS NULL
    ) AS invalid_suppliers
FROM clean.fact_bookings AS b
LEFT JOIN clean.dim_agents AS a ON b.agent_id = a.agent_id
LEFT JOIN clean.dim_hotels AS h ON b.hotel_id = h.hotel_id
LEFT JOIN clean.dim_destinations AS d ON b.destination_id = d.destination_id
LEFT JOIN clean.dim_suppliers AS s ON b.supplier_id = s.supplier_id;
```

Check	Verified result	Meaning
fact_bookings	119,877	Verified
fact_cancellations	13,972	Verified
dim_agents	8,000	Verified
dim_hotels	3,000	Verified
dim_suppliers	160	Verified
dim_destinations	280	Verified
Serious invalid rules in fact_bookings	0 / 0 / 0 / 0	Verified
Payment summary final row count	Re-run final validation	The lesson was taught, but your final result was not posted in chat

Phase 2 completion gate Do not move to Phase 3 until the payment summary count, orphan-payment checks and final relationship query are run successfully in your current database.

12. Phase 2 Results Summary

Check	Verified result	Meaning
Raw bookings	120,175	Preserved unchanged
Deduplicated bookings	120,000	175 extra duplicate copies removed from clean path
Serious invalid bookings	123	Documented in exceptions table
Final clean booking fact	119,877	Unique, serious-rule-valid bookings
Missing supplier IDs	240	Retained with a warning flag
Missing agent references	0	Verified after reconciliation
Clean agents	8,000	Unique agent IDs
Clean hotels	3,000	Unique hotel IDs; 35 missing ratings remain NULL
Clean suppliers	160	Unique supplier IDs
Clean destinations	280	Unique destination IDs
Raw cancellations	13,987	Unique cancellation IDs
Orphan cancellations	15	Excluded from clean cancellation fact
Clean cancellations	13,972	No orphan cancellations remain
Payment transactions	127,189	Source package count; aggregate before booking join

13. Concepts You Learned in Phase 2

Concept	Beginner explanation
Layered data architecture	Separate original source data, transformation logic and trusted analysis data.
Safe type conversion	Validate text with patterns before casting to DATE or NUMERIC.
Standardization	Use TRIM, UPPER, INITCAP and CASE to create consistent category values.
Window functions	ROW_NUMBER() with PARTITION BY identifies duplicate copies within a business key.
Data-quality flags	Binary 0/1 fields make issues visible and auditable.
Exception tables	Preserve excluded records with reasons instead of silently deleting them.
Fact table grain	One row per business event at a declared level - here, one row per valid booking.
Dimension table grain	One row per descriptive entity such as agent, hotel, supplier or destination.
Referential integrity	Foreign keys should match dimension IDs; missing relationships must be measured.
One-to-many joins	A booking can have multiple payment rows, so direct joins can duplicate booking metrics.
Aggregation before joining	Summarize payments to booking grain before combining them with booking KPIs.
Reconciliation	Conflicting counts must be rechecked against source and transformed layers before reporting.
Defensive SQL	NULLIF, CASE, COALESCE and validation queries reduce silent analytical errors.
Traceability	Every cleaned result should be explainable back to raw values and business rules.

14. How to Decide What to Keep, Flag, Exclude or Investigate

Issue	Treatment	Reason
Exact duplicate booking copy	Exclude extra copy from clean fact	Same business event counted twice would inflate KPIs
rooms <= 0	Exception / exclude from main KPI fact	Booking quantity is structurally invalid
nights <= 0	Exception / exclude	Stay duration is structurally invalid
GBV <= 0	Exception / exclude	Main commercial amount is invalid for standard KPI layer
travel date < booking date	Exception / exclude	Impossible or suspicious booking sequence
supplier_id missing	Keep + flag	Booking still supports many non-supplier analyses
star rating missing	Keep NULL	Do not invent descriptive values
orphan cancellation	Exclude from clean cancellation fact, preserve raw	No parent booking exists
multiple payment rows	Keep all transactions; aggregate for booking joins	Multiple transactions can be legitimate
category capitalization differences	Standardize	Same real category should not split into multiple labels

15. Common Mistakes to Avoid

Deleting from raw tables: Never “clean” by updating or deleting the original imported source unless the project explicitly requires it.

Casting raw text blindly: A single nonnumeric value can crash a cast. Pattern-check or use controlled CASE logic.

Removing all rows with missing values: A missing supplier does not make every booking metric unusable.

Joining payments directly to bookings: This can repeat booking GBV and cause double counting.

Assuming table creation means success: Always validate row counts, NULLs, unique keys and business rules.

Using manually typed labels as evidence: Profile the values stored in the database; user-entered notes can contain typos.

Summing mixed currencies: Phase 2 does not solve FX. Monetary company-wide totals must wait for Phase 3.

Ignoring inconsistent historical results: Reconcile them. The corrected agent-reference example is a good portfolio story about QA discipline.

16. Reusable Phase 2 Checklist

1. 1. Create staging and clean schemas.
2. 2. Create typed booking staging table and validate conversion failures.
3. 3. Standardize categorical text values while preserving raw values.
4. 4. Rank duplicate business keys with ROW_NUMBER().
5. 5. Create one-row-per-booking deduplicated data.
6. 6. Build serious data-quality flags.
7. 7. Create an exception table for excluded records.
8. 8. Differentiate “invalid booking” from “incomplete dimension mapping.”
9. 9. Create the main clean booking fact at a declared grain.
10. 10. Validate primary-key uniqueness in every dimension.
11. 11. Standardize and create clean agent, hotel, supplier and destination dimensions.
12. 12. Clean cancellation events and remove true orphans from the clean fact.
13. 13. Type payment transactions and aggregate them to booking grain.
14. 14. Check payment orphans before downstream joins.
15. 15. Run final row-count, uniqueness, business-rule and relationship reconciliations.
16. 16. Do not create company-wide money KPIs until currencies are normalized.

17. How to Explain Phase 2 in an Interview

60-second explanation I preserved the raw travel-booking source, created staging tables to safely type dates and money fields, standardized categorical labels, ranked and removed duplicate booking copies, and built data-quality flags and an exception table. I then created a one-row-per-booking clean fact and unique agent, hotel, supplier and destination dimensions. I cleaned cancellation events, removed orphan records, and aggregated payment transactions to booking grain to prevent double counting. Finally, I reconciled counts and business rules before moving to FX normalization.

Interview questions you should be able to answer

- Why did you preserve the raw layer instead of updating it?
- What is the difference between staging and clean?
- How did you safely convert mixed-format dates?
- Why did you use ROW_NUMBER() for deduplication?
- Why did you keep missing supplier bookings instead of deleting them?
- What is an exception table and why is it useful?
- What is the grain of fact_bookings?
- Why must dimension IDs be unique before a join?
- Why can a direct bookings-to-payments join double count GBV?
- How did you validate that your clean layer was trustworthy?
- Why are Phase 2 revenue totals still unsafe across currencies?

18. Quick SQL Reference

Safe numeric cast

```
CASE
    WHEN TRIM(value_col) ~ '^-?\d+(\.\d+)?$'
    THEN value_col::numeric
    ELSE NULL
END
```

Safe mixed-format date cast

```
CASE
    WHEN date_col ~ '^\d{4}-\d{2}-\d{2}$' THEN date_col::date
    WHEN date_col ~ '^\d{2}/\d{2}/\d{4}$' THEN TO_DATE(date_col,'DD/MM/YYYY')
    ELSE NULL
END
```

Deduplicate by business key

```
ROW_NUMBER() OVER (
    PARTITION BY booking_id
    ORDER BY booking_id
) AS duplicate_rank
```

Conditional count

```
COUNT(*) FILTER (WHERE condition)
```

Find orphan foreign keys

```
SELECT child.*
FROM child_table AS child
LEFT JOIN parent_table AS parent
    ON child.parent_id = parent.parent_id
WHERE parent.parent_id IS NULL;
```

Aggregate one-to-many data before joining

```
SELECT
    booking_id,
    COUNT(*) AS transaction_count,
    SUM(amount) AS total_amount
FROM payments
GROUP BY booking_id;
```

19. Phase 2 Completion and Phase 3 Handoff

After the final payment and relationship validations pass, Phase 2 is complete. The clean layer now provides controlled grains, standardized categories, documented exceptions and safe joins. The next problem is financial comparability: the bookings still use multiple transaction currencies.

Next phase Phase 3 - FX Normalization: create exchange-rate logic, convert booking/payment/cancellation money into a common reporting currency (INR in this portfolio project), validate FX coverage, and only then build company-wide revenue analytics.

What you should remember most

- Raw preserves.
- Staging prepares.
- Clean trusts.
- Exceptions explain exclusions.
- Grain controls joins.
- Validation creates confidence.
- Currency normalization comes before company-wide monetary KPIs.